Data Structures (ETCCDS202)

Assignment - 3

Name of the School: School of Engineering & Technology

Program: B. Tech CSE (AI & ML)

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Sorting Algorithms

Student Name: Abhay Kumar

Roll No.: 2501730185

Section: A

1. Introduction

Sorting is a fundamental operation in computer science used to arrange data in a specific order, typically in ascending or descending order. Different sorting algorithms exhibit different performance characteristics depending on the nature and size of the input data.

This report focuses on understanding and comparing three important sorting algorithms:

- Insertion Sort
- Merge Sort
- Quick Sort

The goal is to study their working principles, theoretical time complexities, stability, and memory usage.

2. Algorithms Overview

2.1 Insertion Sort

Insertion Sort builds the sorted array one element at a time by inserting each element into its correct position.

Working Principle:

- Start from the second element.
- Compare it with previous elements.
- Shift larger elements to the right.
- Insert the element in the correct position.

Complexity:

- Best Case: $O(n)$ (already sorted)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$ (reverse sorted)

Properties:

- Stable: Yes
- In-place: Yes
- Efficient for small or nearly sorted datasets

2.2 Merge Sort

Merge Sort is a divide-and-conquer algorithm that divides the array into smaller subarrays, sorts them, and then merges them.

Working Principle:

- Divide the array into two halves.
- Recursively sort both halves.
- Merge the sorted halves.

Complexity:

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)

Properties:

- Stable: Yes
- In-place: No
- Requires additional memory for merging

2.3 Quick Sort

Quick Sort is also a divide-and-conquer algorithm that works by selecting a pivot element and partitioning the array around it.

Working Principle:

- Choose a pivot element.
- Place smaller elements to the left and larger to the right.
- Recursively apply the same process to subarrays.

Complexity:

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$ (when partitions are unbalanced)

Properties:

- Stable: No
- In-place: Yes
- Very efficient in practice for large datasets

3. Theoretical Comparison

Time Complexity Comparison

- Insertion Sort performs well only for small or nearly sorted data.
- Merge Sort guarantees $O(n \log n)$ performance in all cases.
- Quick Sort is generally fast but can degrade to $O(n^2)$ in worst cases.

Space Complexity

- Insertion Sort: $O(1)$ (constant extra space)
- Merge Sort: $O(n)$ (extra memory required)
- Quick Sort: $O(\log n)$ (recursion stack)

4. Stability and In-Place Behavior

Algorithm	Stable	In-place	Remarks
Insertion Sort	Yes	Yes	Simple and efficient for small data
Merge Sort	Yes	No	Uses extra memory but consistent
Quick Sort	No	Yes	Fast but unstable

5. Discussion

- **Insertion Sort** is useful when the dataset is small or nearly sorted. Its simplicity makes it easy to implement, but it becomes inefficient for large datasets.
- **Merge Sort** provides consistent performance regardless of input type. It is particularly useful when stability is required, but its extra space requirement is a disadvantage.
- **Quick Sort** is widely used due to its average-case efficiency and in-place nature. However, its performance depends heavily on pivot selection. Poor pivot choices can lead to worst-case behavior.

6. Special Note on Quick Sort

When the pivot is chosen poorly (for example, always choosing the last element in an already sorted array), Quick Sort produces highly unbalanced partitions. This results in:

- Increased recursion depth
- Degraded performance to $O(n^2)$

This issue can be mitigated using better pivot selection strategies such as:

- Random pivot
- Median-of-three method

7. Conclusion

- Insertion Sort is best suited for small or nearly sorted datasets.
- Merge Sort is reliable and stable with guaranteed performance.
- Quick Sort is efficient in practice but sensitive to pivot selection.

Overall, the choice of sorting algorithm depends on the input data characteristics and memory constraints.